GAME HUB · ARCHITECTURE

External Services & Libraries

Networking

- PeerJS cloud (matchmaking / signalling)
- TURN relay (optional, your own credentials)

Accounts & Data

- Firebase Authentication (e-mail + password)
- Cloud Firestore (invites, users, activity, recovery)

E-mail

- EmailJS (secret-draw and invite e-mails)

Libraries & CDN

- PeerJS library (unpkg / jsDelivr)
- Firebase JS SDK (gstatic.com)
- Google Fonts (Fredoka)

User Interaction Layer

- **Players** (browser / phone)
- **Web UI** — single page (index.html)
- **Sign in / Register** — invite-only · forgot password
- **My Activity** — personal stats & recovery question
- **Admin Dashboard** — people, invites, activity & roles

Game Layer

Game Host Engine – runs in the host's browser (the host is the single source of truth)

- **Number Game** — N / P digit clues
- **Who Is the Spy?** — roles, clues, votes
- **Raja Rani** — secret roles, Police guess
- **Scribblio** — draw & guess, live canvas
- **Chess** — 2 / 3 / 4 players, computer, clocks
- **Dots and Boxes** — dice, lines, boxes
- **Ludo** — 4 / 6 colours, computer seats
- **Rock Paper Scissors** — duel, knockout, computer

Room state: players · settings · rounds · scores · timers (500 ms host tick) · kick / leave handling

Computer players (chess engine, Ludo bot, Rock Paper Scissors) are run by the host too

Views & Events

- Per-player views (secrets never leave the host)
- Ejection & role cut-scenes (with sound)
- Toasts, kick / leave notices, results

Networking Layer

- **Room code & invite link** — 5-letter code · ?room=CODE
- **PeerJS host (WebRTC)** — guests ↔ host star topology
- **Messages** — join · state · act · draw · toast · kicked
- **Reconnect & ICE** — auto-rejoin on refresh, STUN / optional TURN

Accounts & Data Layer

- **Firebase Auth** — e-mail + password
- **Invite check** — only invited e-mails can register
- **Activity tracking** — logins, games, play time, online presence
- **Password recovery** — security question, no e-mail needed

Cloud Firestore

- **invites/<email>** — status: invited / registered / removed
- **users/<uid>** — profile, role, last sign-in, stats
- **users/<uid>/activity** — sign-ins and games played
- **recovery/<email>** — encrypted password record

Infrastructure Layer

- **GitHub Pages** — static hosting (anchana-techie.github.io)
- **Service Worker** — offline cache (sw.js)
- **Browser storage** — local / session storage
- **Firebase project** — Auth + Firestore (Spark plan)
- **PeerJS cloud** — free matchmaking server

Cross-Cutting Services

- Authentication & Authorization (invite-only, roles)
- Activity tracking & presence
- Security rules (Firestore)
- Offline & error pages, auto-reconnect

Outputs

- Game results, scores & rankings
- Admin insights (people & activity)
- Personal activity statistics
- Secret-draw & invite e-mails

Game Hub – Architecture & Documentation

Game Hub is a static web app (no build step, no application server). Everything runs in the browser; the only cloud services are the free PeerJS matchmaking server, Firebase (accounts + Firestore) and optionally EmailJS.

1. Layers at a glance

Layer	What it does	Where in the code
User Interaction	Home screen, sign-in / register, room lobby, game screens, My Activity, Admin dashboard, offline and system pages	`landingHTML`, `entryHTML`, `roomHTML`, `*GameHTML`, `authHTML`, `adminHTML`, `myStatsHTML`, `offlineHTML`
Game	The **host's browser** is the game server: it owns the room, runs the rules, timers and computer players, and sends every player a personal view	`doAct`, `deal`, `hostTick`, `viewFor` / `gameView`, `*Deal / *Act / *Tick` per game
Networking	Room code / invite link, WebRTC connections through PeerJS, message passing, reconnect	`createRoom`, `joinRoom`, `dispatch`, `pushState`, `iceConfig`
Accounts & Data	Invite-only sign-in, roles, activity tracking, password recovery	`fb*`, `auth*`, `adm*`, `track*`, `rc*`, `firestore.rules`
Infrastructure	GitHub Pages hosting, service worker cache, browser storage, Firebase project, PeerJS cloud	`sw.js`, `firebase-config.js`

Cross-cutting: authentication / authorisation (invite-only, `role: "admin"`), activity tracking and presence, Firestore security rules, offline / error pages and automatic reconnect.

2. Games

Game	Players	Notes
🎯 Number Game	1–10	N / P digit clues, rounds, timers; also playable offline
🕵️ Who Is the Spy?	3–15	secret words and roles, clue cycle, voting
👑 Raja Rani	3–20	secret roles, Police guesses the Thief
🎨 Scribblio	2–12	live drawing canvas, guessing chat, letter hints
♟️ Chess	1–4 seats	classic, 3-player hexagon, 4-player cross; computer (3 levels), clocks; offline capable
🎲 Dots and Boxes	2–20	dice, lines, boxes stamped with emoji
🏁 Ludo	up to 6	4- or 6-colour board, computer or empty colours, burning movable coins
✊ Rock Paper Scissors	up to 10	duel, knockout, computer players, astronaut "hi" on the reveal
🎄 / 🪔 ChristMom, Diwali	any	secret matching by e-mail (EmailJS)

Details of each game's rules are in README.md.

3. How a game runs

1. **Create / join.** The host creates a room (a 5-letter code, PeerJS id `wfg-<CODE>`); guests open the invite link (`?room=CODE`) or type the code.
2. **Lobby.** The host changes settings (`SETTINGS` / `DEFAULTS`) and starts the game (`startGame` → `deal`).
3. **Play.** A player's click becomes an *action* (`sendAct`). A guest sends `{t:"act"}` to the host; the host runs `dispatch → doAct →` the game's `*Act` function and then `pushState`, which sends every player a **personal view** (`viewFor`). Secrets (words, roles, hidden throws) are filtered out of other players' views.
4. **Timers & computers.** `hostTick` runs every 500 ms: it handles time-outs, computer moves (chess engine, Ludo bot, Rock Paper Scissors) and disconnected players.
5. **Reveal → next round → end.** Phases are `lobby → play → end` for the room and `… → reveal` inside a game. The host presses *Next round*, *New game* or *Back to lobby*.
6. **Leaving.** A player who leaves or is kicked is out: Ludo tokens and chess pieces are removed, a see-through airlock cut-scene plays, and games that fall below their minimum player count stop.

Messages between peers

Direction	Message	Purpose
guest → host	`join` (pid, name, avatar)	enter the room
guest → host	`act` (`m`)	any game action
guest → host	`draw`	Scribblio strokes
host → guest	`state` (`s`)	the player's personal view
host → guest	`toast`	a private notice or error
host → guest	`kicked`	removed by the host
host → guest	`draw`	relayed Scribblio strokes

If players can't connect (strict networks), *Connection help* accepts a TURN relay's credentials (`iceConfig`).

4. Accounts, invites and admin (Firebase)

Enabled only when `firebase-config.js` contains real keys; otherwise the hub runs without login.

Sign-up rules

- Only an e-mail the admin has **invited** can register – checked by the app and by Firestore rules.
- Sign-in is e-mail + password. E-mails are **not** verified (they are only IDs).
- **Forgot password** uses a **security question** chosen at registration: the password is stored encrypted under a key derived from the answer (PBKDF2 → AES-GCM) in `recovery/<email>`. A strong answer matters.
- A person removed by the admin loses access immediately, can't register again until re-invited, and their leftover sign-in record is deleted the next time they try to sign in.

Roles

- Roles are stored on the profile: `users/<uid>.role` is `"user"` or `"admin"`.

- The first admin is created by hand in the Firebase console (see FIREBASE_SETUP.md); admins can then promote or demote others in the Admin dashboard.

Firestore data model

Collection	Key	Fields
`invites`	`<email>`	`email`, `name`, `status` (`invited` / `registered` / `removed`), `invitedAt`, `invitedBy`, `uid`, `registeredAt`, `removedAt`
`users`	`<uid>`	`email`, `name`, `role`, `createdAt`, `lastLoginAt`, `lastSeen`, `online`, `loginCount`, `device`, `gamesPlayed`, `totalSeconds`, `lastGame`, `lastPlayedAt`, `stats.<game> = played, hosted, completed, wins, score, seconds, last`
`users/<uid>/activity`	auto id	`type` (`login` / `game`), `at`, `device`, `first`; for games: `game`, `role` (host / player), `durationSec`, `completed`, `won`, `score`, `players`, `room`
`recovery`	`<email>`	`q` (question), `salt`, `iv`, `ct`, `iter`, `updatedAt`

Activity tracking

Each signed-in browser records its own data: a sign-in per tab session, a 60-second heartbeat (`lastSeen`, `online`), and per game: start (`played`, `hosted`), end (`completed`, `wins`, `score`, `seconds`) and an activity entry (`trackRoom` / `trackEnd`). Admins read all of it live (`onSnapshot`) in the Admin dashboard.

Admin dashboard

Summary cards (accounts, invited, online now, active in 7 days, games played, play time, most played game), invite form (with copy-link and optional e-mail through EmailJS), and a searchable People table: status, last sign-in, sign-ins, games and play time, device, with per-game statistics and recent activity per person. Actions: invite, revoke, re-invite, remove, make / remove admin.

Security rules (`firestore.rules`)

- `invites`: anyone may read one invite by e-mail (to validate registration); only admins list, create or delete; a person may flip their own invite `invited → registered`.
- `recovery`: readable by e-mail while signed out; only that person (or an admin) can write or delete it.
- `users`: a person reads and updates their own profile (but cannot change `role` or `email`); creating a profile requires an `invited` invite and `role: "user"`; admins read, update and delete all.
- `users/<uid>/activity`: the owner creates and reads; admins read and delete.

5. Offline and caching

- `sw.js` (service worker): pages are network-first with a saved copy, libraries and fonts come from the cache and refresh in the background; PeerJS signalling, EmailJS and connectivity probes are never touched.
- Chess, the Number Game, Ludo and Rock Paper Scissors can run on one device with no internet (`OFFLINE_OK`, `createLocalRoom`). If the Firebase scripts can't load while offline the hub runs signed-out.
- System pages (`#/loading`, `#/error`, `#/no-internet`, `#/404`, `#/offline-mode`) cover start-up, errors and no connection.

6. Browser storage

Key	Content
`localStorage ngg_me`	player name and avatar
`localStorage ngg_turn`, `xm_emailjs`	TURN relay and EmailJS settings
`localStorage ngg_mute`	cut-scene sound toggle
`sessionStorage ngg_pid`, `ngg_room`	this tab's player id and the room to rejoin after a refresh
`sessionStorage gh_login_<uid>`	marks that this tab's sign-in was already counted

7. Files

File	Purpose
`index.html`	the entire app (HTML, CSS, JavaScript)
`sw.js`	service worker for offline use
`offline.html`, `404.html`	fallback and not-found pages
`firebase-config.js`	Firebase web keys (public by design)
`firestore.rules`	Firestore security rules – publish in the Firebase console
`FIREBASE_SETUP.md`	step-by-step Firebase setup
`docs/architecture.svg`, `docs/ARCHITECTURE.md`	this documentation

8. Limits to know about

- The **host's browser is the server**: if the host closes the tab, the room ends.
- Game statistics are written by each player's browser – fine for friends, not tamper-proof.
- Deleting another person's Firebase sign-in record needs Cloud Functions / the Admin SDK, which this project does not use.
- Free Firebase (Spark) plan and the free PeerJS server are enough; there are no paid components.